

INFORME TÉCNICO: TRABAJO PRÁCTICO N° 3 – DESARROLLO DEL ABM COMPLETO

Institución: IES21

Materia: Aplicaciones Web II

Profesor: Andrés Senn

Integrantes: Luján Federico, Giménez Fabrizio

1. Introducción

El presente informe técnico expone el proceso de diseño, desarrollo e integración del módulo de administración completa (ABM/CRUD) para el sitio web de la rotisería **"Como en casa"**. Esta etapa representó el núcleo de la materia, ya que implicó la unificación definitiva de la interfaz de usuario (Frontend) con la lógica del servidor (Backend) y el motor de persistencia.

A lo largo del desarrollo, el equipo dedicó un tiempo considerable al análisis y comprensión de la arquitectura cliente-servidor. Comprender el flujo asíncrono de los datos, el ciclo de vida de una petición HTTP y la manipulación dinámica del DOM mediante JavaScript demandó un proceso de aprendizaje intensivo, lo que ralentizó inicialmente el avance pero garantizó una base sólida para lograr un sistema robusto, óptimo y libre de fallos estructurales.

2. Elección Tecnológica: PostgreSQL como Motor de Persistencia

Para este proyecto se optó de manera unánime por utilizar **PostgreSQL** como Sistema de Gestión de Bases de Datos Relacionales (RDBMS), descartando otras alternativas del mercado. Los fundamentos técnicos que justifican esta elección son:

- **Robustez y Cumplimiento ACID:** Al tratarse de un sistema de gestión de menús y pedidos, garantizar la atomicidad, consistencia, aislamiento y durabilidad de las transacciones es crítico para que los datos jamás se corrompan.
- **Manejo Eficiente de Tipos de Datos:** PostgreSQL ofrece un excelente soporte para tipos de datos numéricos exactos (**NUMERIC**), ideales para almacenar precios de productos sin sufrir los errores de redondeo que poseen los tipos flotantes en otros motores.
- **Escalabilidad y Concurrencia:** Mediante el uso de un pool de conexiones en Node.js (**pg.Pool**), el servidor puede atender múltiples peticiones concurrentes desde la interfaz administrativa de forma eficiente, reutilizando los canales de comunicación abiertos con la base de datos.

3. Arquitectura del Sistema y Flujo de Datos (La Lógica Comprendida)

Uno de los mayores desafíos del equipo fue asimilar cómo interactúan las capas del proyecto de forma asíncrona a través del protocolo HTTP. El flujo implementado se estructuró bajo el siguiente estándar lógico:

1. **El Frontend solicita o envía datos:** Mediante la API **fetch** de JavaScript, la interfaz administrativa realiza peticiones hacia endpoints específicos (ej. **/api/productos**).

2. **El Enrutador del Backend procesa la solicitud:** Express intercepta el método HTTP (**GET**, **POST**, **PUT** o **DELETE**) y el parámetro (ID) si corresponde.
3. **El Modelo ejecuta la consulta SQL:** A través del archivo **modelo.productos.mjs**, se despachan las sentencias SQL puras utilizando el cliente de PostgreSQL.
4. **La Base de Datos responde y el Frontend renderiza:** Los registros retornados por la base de datos se transforman en formato JSON en el servidor, viajan de vuelta por la red, y el Frontend se encarga de limpiar el contenedor HTML y dibujar las tarjetas dinámicamente mapeando las propiedades de cada objeto.

4. Estructura de la Tabla y Optimización SQL

Se definió la estructura de la tabla **menu** garantizando que cada registro posea una identidad única y campos estrictamente tipados.

SQL

```
CREATE TABLE menu(  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100),  
  descripcion TEXT,  
  imagen VARCHAR(255),  
  precio NUMERIC(10,2)  
);
```

Optimización del Listado General

Durante las pruebas de modificación de productos, el equipo detectó un comportamiento nativo de PostgreSQL: al realizar un **UPDATE**, el motor reubica físicamente la fila modificada al final de la tabla. Para mitigar este impacto visual y evitar que los productos se desordenaran aleatoriamente en la pantalla del administrador, se

optimizó la consulta del método `GET` añadiendo un ordenamiento estricto:

SQL

```
SELECT * FROM menu ORDER BY id ASC;
```

Esto asegura que la interfaz mantenga siempre una estructura visual idéntica y predecible para el usuario, independientemente de las ediciones que se realicen.

5. Detalle Fino de las Funcionalidades Desarrolladas

A. Listado Automatizado (Lectura - GET)

Al inicializarse el módulo de administración, se dispara la función asíncrona `cargarProductos()`. Esta realiza un `fetch` hacia la API, recibe el array de productos ordenados y recorre la estructura mediante un ciclo `forEach`, inyectando los nodos de HTML dinámico con las rutas relativas correspondientes para renderizar las imágenes locales (`./recursos/imagenes/`).

B. Alta de Productos (Escritura - POST)

Se capturó el evento `submit` del formulario. Se previene la recarga por defecto de la página (`e.preventDefault()`), se extraen los valores de los inputs y se empaquetan en un objeto JSON. Si el servidor procesa exitosamente la inserción en la base de datos, el formulario se resetea por completo (`e.target.reset()`) y se vuelve a invocar de manera transparente el listado actualizado.

C. Modificación Bidireccional (Actualización - PUT)

La lógica de la modificación requirió un control de estados en el Frontend mediante una variable global (`editandoId`). El proceso se divide en:

- **Fase de Carga:** Al presionar "Modificar", se busca el producto por su ID mediante la API, se inyectan sus valores actuales directamente en los inputs del formulario y se cambia el texto del botón principal a "Guardar Cambios".
- **Fase de Persistencia:** Cuando el usuario envía el formulario con las modificaciones, el código evalúa si `editandoId` contiene un valor. De ser así, redirige la petición utilizando el método `PUT` hacia `/api/productos/:id`. El backend procesa los nuevos datos en la base de datos, se limpia el estado (`editandoId = null`) y el botón retorna a su etiqueta original de "Agregar Producto".

D. Baja Segura de Registros (Eliminación - DELETE)

Para prevenir la pérdida accidental de información crítica del menú, se integró una capa de validación en el cliente mediante una ventana modal de confirmación (`confirm`). La función evalúa la respuesta del administrador: si la acción es cancelada, se interrumpe la ejecución del hilo de JavaScript (`return`), protegiendo la integridad del registro. Si la acción es afirmativa, se despacha la petición `DELETE` hacia el servidor, removiendo físicamente la fila en PostgreSQL y actualizando la interfaz instantáneamente.

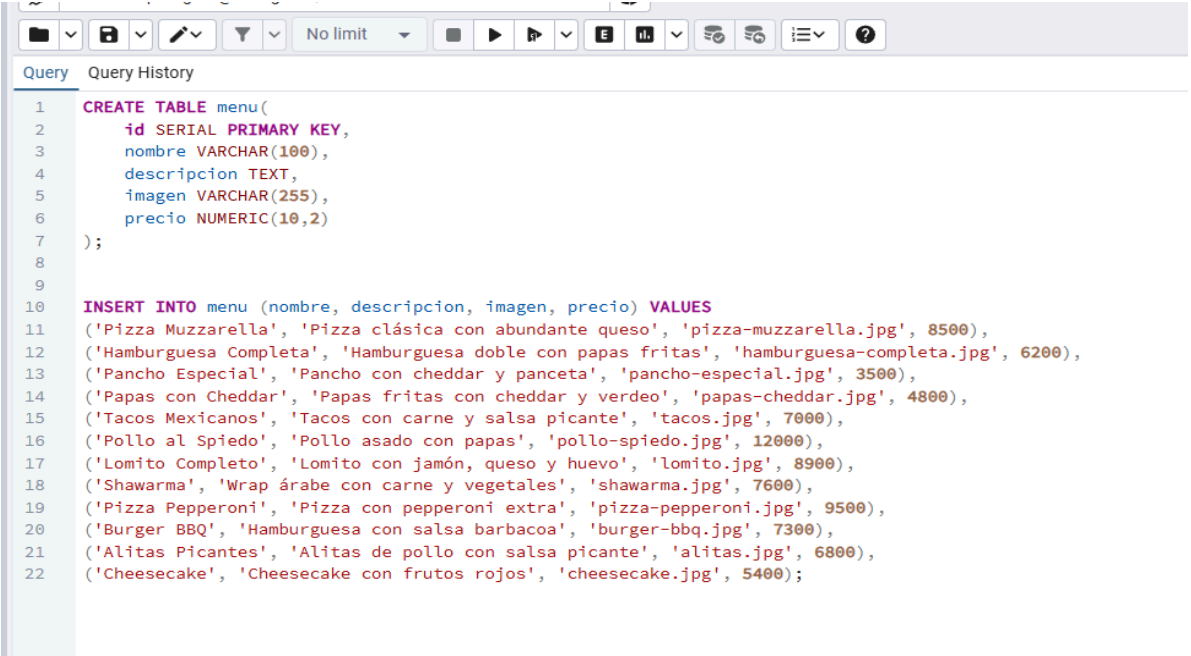
(Nota técnica: Durante el testeo en entornos de producción local, se identificó que las políticas de almacenamiento en caché de ciertos navegadores web ejecutaban versiones obsoletas del script de JavaScript. Esta eventualidad fue sorteada de forma exitosa forzando la limpieza de la memoria caché del navegador e

implementando entornos cerrados de pruebas en modo incógnito, garantizando la lectura en caliente del código actualizado).

6. Sección de Evidencias Técnicas (Capturas a Incluir)

(Espacio dispuesto para el pegado de las capturas del software en ejecución)

1. Captura del Script de Inserción:



```
1 CREATE TABLE menu(  
2     id SERIAL PRIMARY KEY,  
3     nombre VARCHAR(100),  
4     descripcion TEXT,  
5     imagen VARCHAR(255),  
6     precio NUMERIC(10,2)  
7 );  
8  
9  
10 INSERT INTO menu (nombre, descripcion, imagen, precio) VALUES  
11 ('Pizza Muzzarella', 'Pizza clásica con abundante queso', 'pizza-muzzarella.jpg', 8500),  
12 ('Hamburguesa Completa', 'Hamburguesa doble con papas fritas', 'hamburguesa-completa.jpg', 6200),  
13 ('Pancho Especial', 'Pancho con cheddar y panceta', 'pancho-especial.jpg', 3500),  
14 ('Papas con Cheddar', 'Papas fritas con cheddar y verdeo', 'papas-cheddar.jpg', 4800),  
15 ('Tacos Mexicanos', 'Tacos con carne y salsa picante', 'tacos.jpg', 7000),  
16 ('Pollo al Spiedo', 'Pollo asado con papas', 'pollo-spiedo.jpg', 12000),  
17 ('Lomito Completo', 'Lomito con jamón, queso y huevo', 'lomito.jpg', 8900),  
18 ('Shawarma', 'Wrap árabe con carne y vegetales', 'shawarma.jpg', 7600),  
19 ('Pizza Pepperoni', 'Pizza con pepperoni extra', 'pizza-pepperoni.jpg', 9500),  
20 ('Burger BBQ', 'Hamburguesa con salsa barbacoa', 'burger-bbq.jpg', 7300),  
21 ('Alitas Picantes', 'Alitas de pollo con salsa picante', 'alitas.jpg', 6800),  
22 ('Cheesecake', 'Cheesecake con frutos rojos', 'cheesecake.jpg', 5400);
```

2. Captura del Formulario de Edición Activo:

Administración de Productos

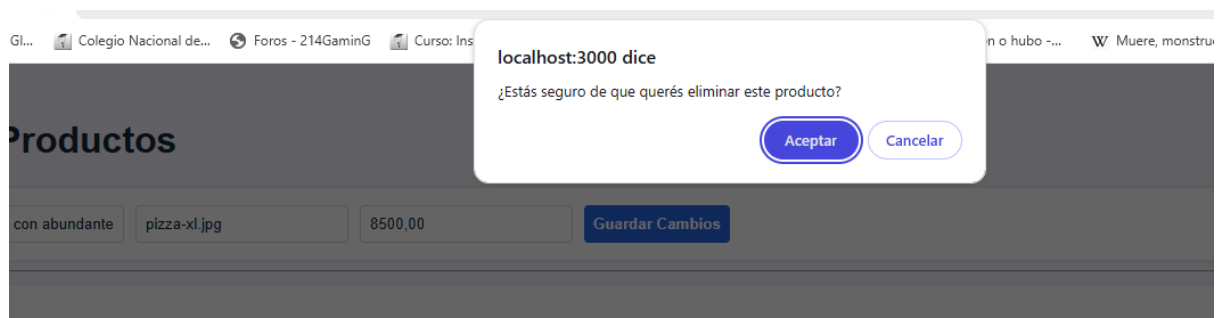
Pizza Muzzarella	Pizza clásica con abundante	pizza-xl.jpg	8500,00	Guardar Cambios
------------------	-----------------------------	--------------	---------	-----------------



Pizza Muzzarella
Pizza clásica con abundante queso
\$8500.00

[Modificar](#) [Eliminar](#)

3. Captura del Mensaje de Confirmación:



7. Conclusión

El desarrollo de este Trabajo Práctico permitió consolidar los conocimientos teóricos sobre el backend, las bases de datos relacionales y el comportamiento síncrono/asíncrono de la web. A pesar de la curva de aprendizaje que significó asimilar la lógica de integración de cada componente, el resultado es un sistema administrativo ABM eficiente, seguro y altamente escalable para la

rotisería "**Como en casa**", cumpliendo rigurosamente con los estándares solicitados por la cátedra.